

Base de Datos II

TP 7: Drivers, Conexión y Modelado con ODM

Joaquín VIllalba - joaquinvillalba8@gmail.com

17/10/2026

Introducción Teórica

En el desarrollo de aplicaciones con MongoDB, existen dos enfoques principales para gestionar la comunicación:
1. Driver Nativo: Provee la comunicación más directa con el motor de base de datos. Se destaca por ofrecer el máximo control sobre las consultas y una mayor eficiencia al no tener capas intermedias.
2. Mongoose (ODM): Actúa como una capa de abstracción sobre el driver. Permite definir esquemas para organizar los datos y forzar validaciones directamente en el código de la aplicación.

Parte 1: Implementación con Driver Nativo

El driver oficial (mongodb) es ideal para aplicaciones sencillas o procesos de alto volumen donde el rendimiento es crítico.

Instrucciones:
1. Inicialicen el proyecto e instalen el paquete: npm install mongodb.
2. Creen un archivo driver_lab.js para conectarse a la base de datos laboratorio.
3. Usando el método insertMany, carguen un array con tres sensores de robótica (ej: Sensor de Humedad, Servo Motor, LED RGB).

```
PS C:\Users\JNV\Documents\UTN\2026\4to Cuatri\BD 2\7 - Conexión entre aplicaciones y
MongoDB\tp7>node driver_lab.js
Sensores insertados: 3
IDs: {
  '0': new ObjectId('6ad3d0c8e51cf39c384f82e9'),
  '1': new ObjectId('6ad3d0c8e51cf39c384f82ea'),
  '2': new ObjectId('6ad3d0c8e51cf39c384f82eb')
}
```

Parte 2: Implementación con Mongoose (ODM)

Mongoose ayuda a mantener el orden y la coherencia en aplicaciones de mediana o gran escala.

Instrucciones:
1. Instalen el paquete: npm install mongoose.
2. Definan un Esquema (Schema) para los componentes que incluya:
○ nombre: Tipo String y obligatorio (required: true).
○ stock: Tipo Number.
3. Activen la opción timestamps: true para generar automáticamente los campos de fecha de creación y actualización.
4. Creen el modelo y realicen una inserción de prueba.

```
PS C:\Users\JNnode mongoose_lab.js\4to Cuatri\BD 2\7 - Conexión entre aplicaciones y MongoDB\tp7>
Componente insertado: {
  nombre: 'Sensor Ultrasónico',
  stock: 8,
  _id: new ObjectId('6ad3d1a63b73b2466766c440'),
  createdAt: 2026-10-17T19:51:02.326Z,
  updatedAt: 2026-10-17T19:51:02.326Z,
  __v: 0
}
```

Parte 3: Reflexión Final

Basándose en lo aprendido, respondan:

1. ¿Cuál es la función del driver como "traductor" entre la aplicación y MongoDB?.
El driver actúa como intermediario entre la aplicación y MongoDB. Se encarga de gestionar la conexión, realizar la serialización y deserialización de datos y permitir la ejecución de comandos como consultas, inserciones y actualizaciones.

2. ¿A qué formato convierte el driver las estructuras de datos nativas (como objetos de JavaScript) para enviarlas al servidor?.
El driver convierte las estructuras de datos nativas al formato BSON, utilizado por MongoDB para representar e intercambiar datos.

3. Según lo visto, ¿en qué situación de un proyecto de ingeniería elegirían usar el driver nativo en lugar de Mongoose?.
Elegiría el driver nativo para microservicios o aplicaciones sencillas, o en proyectos que procesen grandes volúmenes de datos donde el rendimiento sea crítico. Esto se debe a que permite un acceso más directo a MongoDB, con mayor control sobre las consultas y sin agregar capas adicionales de abstracción.